

Numerical calculation of stochastic impedance

Branko Meeus

March 31, 2025

1 General theory

The theory as laid out in this section is taken from [4] and is shortened to include only the relevant parts. For a full treatment of the theory, please refer to the original document.

1.1 Definition of stochastic impedance

We start from a discrete stochastic system with N sites that can be described by a master equation:

$$\dot{\mathbf{P}}(t) = \mathbf{W}(t)\mathbf{P}(t). \quad (1)$$

with $\mathbf{P}(t)$ a probability vector and $\mathbf{W}(t)$ is a stochastic $N \times N$ matrix whose elements $W_{mn}(t)$ ($m \neq n$) are the transition rates to go from state n to state m at time t and $W_{nn}(t) = -\sum_{m \neq n} W_{mn}(t)$. The system is assumed to satisfy the following conditions:

- (i) in the absence of an external driving, the system evolves towards a unique state of thermal equilibrium;
- (ii) for those transitions which are affected by the driving, the rates depend continuously on the term $Fg(t)$ with F a measure of the strength of the external driving, and $g(t)$ a cosine function with arbitrary phase. We denote the equilibrium value around which the driving oscillates as F_{eq}

We will define the impedance trough calculating the probability currents

$$J_{mn}(t) = W_{mn}(t)P_n(t) - W_{nm}(t)P_m(t). \quad (2)$$

We consider from now on the long-time limit, i.e. the state of the system after all transients have vanished. In this limit, both the solution to Eq. (1) and the currents become periodic in time.

We consider the driving to be small, such that the system only gives a linear response. Because of condition (ii), the first order approximation of the transition matrix becomes:

$$\mathbf{W}(t) \approx \mathbf{W}^{eq} + \overline{\mathbf{W}}Fg(t), \quad (3)$$

where $\mathbf{W}^{eq} = \mathbf{W}(F_{eq})$ is the equilibrium matrix and $\overline{\mathbf{W}} = \frac{d}{dt}W(t)|_{t=0}$. We define the equilibrium state $\mathbf{P}^{eq}$ as the solution to $\mathbf{W}^{eq}\mathbf{P}^{eq} = 0$ and the adiabatic solution $\mathbf{P}^{ad}(t)$ as

the solution to $\mathbf{W}(t)\mathbf{P}^{\text{ad}}(t) = 0$. Following [4] using methods from [2, 3], the first-order solution of the master equation is

$$\mathbf{P}(t) \approx \mathbf{P}^{\text{ad}}(t) - \int_0^\infty d\tau \sum_{k=2}^N \mathbf{v}_k e^{\lambda_k \tau} \langle \mathbf{v}_k | \bar{\mathbf{P}}^{\text{ad}} \rangle F \dot{g}(t - \tau). \quad (4)$$

With inner product

$$\langle \mathbf{v} | \mathbf{w} \rangle = \sum_{n=1}^N \frac{v_n w_n}{P_n^{\text{eq}}}. \quad (5)$$

The matrix $\mathbf{W}^{\text{eq}}$ is Hermitian with respect to this inner product, provided that it satisfies detailed balance [1]. Hence the eigenvectors are orthonormal, so $\langle \mathbf{v}_k | \mathbf{v}_l \rangle = \delta_{kl}$.

Turning back to the currents Eq. (2) we note that the right-hand side of Eq. (4) can be interpreted as an adiabatic part $\mathbf{P}^{\text{ad}}(t)$ and a non-adiabatic part $\mathbf{P}^{\text{nad}}(t)$. Correspondingly the current can also be split as such,

$$J_{mn}(t) = J_{mn}^{\text{ad}}(t) + J_{mn}^{\text{nad}}(t). \quad (6)$$

For the adiabatic part, we find up to first order in F ,

$$J_{mn}^{\text{ad}}(t) = W_{mn}(t) P_n^{\text{ad}}(t) - W_{nm}(t) P_m^{\text{ad}}(t) \quad (7)$$

$$\approx F g(t) (W_{mn}^{\text{eq}} \bar{P}_n^{\text{ad}} + \bar{W}_{mn} P_n^{\text{eq}} - \{m \leftrightarrow n\}), \quad (8)$$

where $\{m \leftrightarrow n\}$ is shorthand for taking the previous two terms in its expression and swapping the indices m and n . The non-adiabatic current reduces to

$$J_{mn}^{\text{nad}}(t) = - \sum_{k=2}^N \langle \mathbf{v}_k | \bar{\mathbf{P}}^{\text{ad}} \rangle (W_{mn}^{\text{eq}} v_{k,n} - W_{nm}^{\text{eq}} v_{k,m}) \int_0^\infty d\tau e^{\lambda_k \tau} F \dot{g}(t - \tau), \quad (9)$$

Writing

$$F g(t) = \text{Re} [\mathbf{F} e^{i\omega t}] \quad ; \quad J_{mn}(t) = \text{Re} [\mathbf{J}_{mn} e^{i\omega t}] \quad (10)$$

we can finally define the impedance $\mathbf{Z}_{mn}$ as the ratio

$$\frac{1}{\mathbf{Z}_{mn}} \equiv \frac{\mathbf{J}_{mn}}{\mathbf{F}} = \sum_{k=1}^N \frac{A_{mn}^{(k)} i\omega}{i\omega - \lambda_k}, \quad (11)$$

with coefficients $A_{mn}^{(k)}$

$$A_{mn}^{(k)} = \begin{cases} W_{mn}^{\text{eq}} \bar{P}_n^{\text{ad}} + \bar{W}_{mn} P_n^{\text{eq}} - W_{nm}^{\text{eq}} \bar{P}_m^{\text{ad}} - \bar{W}_{nm} P_m^{\text{eq}} & k = 1, \\ -\langle \mathbf{v}_k | \bar{\mathbf{P}}^{\text{ad}} \rangle (W_{mn}^{\text{eq}} v_{k,n} - W_{nm}^{\text{eq}} v_{k,m}) & k \geq 2. \end{cases} \quad (12)$$

1.2 Expansion of $\bar{\mathbf{P}}^{\text{ad}}$

To calculate the coefficients $A_{mn}^{(k)}$ and find the impedance, we need to find a workable form of $\bar{\mathbf{P}}^{\text{ad}}$. To do this we expand the vector $\bar{\mathbf{P}}^{\text{ad}}$ in eigenvectors of $\mathbf{W}^{\text{eq}}$ through first-order

perturbation theory

$$\bar{\mathbf{P}}^{\text{ad}} = - \sum_{k=2}^N \frac{\langle \mathbf{v}_k | \bar{\mathbf{W}} \mathbf{P}^{\text{eq}} \rangle}{\lambda_k} \mathbf{v}_k. \quad (13)$$

We can then rewrite parts of the coefficient $A_{mn}^{(1)}$ as

$$W_{mn}^{\text{eq}} \bar{P}_n^{\text{ad}} - W_{nm}^{\text{eq}} \bar{P}_m^{\text{ad}} = \sum_{k=2}^N \frac{\langle \mathbf{v}_k | \bar{\mathbf{W}} \mathbf{P}^{\text{eq}} \rangle}{\lambda_k} (W_{mn}^{\text{eq}} v_{k,n} - W_{nm}^{\text{eq}} v_{k,m}) \quad (14)$$

$$= \sum_{k=2}^N \langle \mathbf{v}_k | \bar{\mathbf{P}}^{\text{ad}} \rangle (W_{mn}^{\text{eq}} v_{k,n} - W_{nm}^{\text{eq}} v_{k,m}) \quad (15)$$

$$= \sum_{k=2}^N A_{mn}^{(k)}, \quad (16)$$

where the second equality follows because of the orthogonality of the eigenvectors.

Using this, we rewrite (11) as

$$\frac{1}{\mathbf{Z}_{mn}(\omega)} = \bar{W}_{mn} P_n^{\text{eq}} - \bar{W}_{nm} P_m^{\text{eq}} + \sum_{k=2}^N \frac{A_{mn}^{(k)} \lambda_k}{i\omega - \lambda_k}, \quad (17)$$

This is the formula that is used in the code.

2 Implementation using NumPy

The calculations above are implemented in a python script that takes as input the matrices $\mathbf{W}^{\text{eq}}$ and $\bar{\mathbf{W}}$ and returns the coefficients $A_{mn}^{(k)}$, the eigenvalues λ_k and the eigenvectors $\mathbf{v}_k$. The full code is listed in section 3, but here we will go over the key calculations.

It is important to keep in mind that python uses *column first* indexing, so for example the element W_{mn}^{eq} normally corresponds to the array element `weq[n, m]`. The code is written such that this indexing is switched after the calculations of the coefficients. This is done for convenience: when building the matrices $\mathbf{W}^{\text{eq}}$ and $\bar{\mathbf{W}}$, the user can use the same indices in python as on paper, so $W_{mn}^{\text{eq}} = \text{weq}[m, n]$. When analyzing the results, the indices are swapped, such that the coefficients `coeff[n, m, :]` correspond to the transition from n to m , simplifying the process of picking out certain transitions by hand.

The code executes the following steps:

1. Calculate the eigensystem of $\mathbf{W}^{\text{eq}}$, define the equilibrium state $\mathbf{P}^{\text{eq}}$ and orthonormalize the eigenvectors.
2. Calculate the expansion coefficients of (13).
3. Construct the 3-dimensional array containing the coefficients $A_{mn}^{(k)}$.

Since the system has a single equilibrium state by assumption, this state is the unique

eigenvector corresponding to $\lambda = 0$ whose entries sum to 0. This means that we do not need to calculate it separately, as we get it for free when determining the eigensystem. In the code this is implemented as:

```
vals, vecs = lin.eig(weq)
vals = np.real(vals)
vecs = np.real(vecs)

ind_arr = np.flip(np.argsort(vals))

vals = vals[ind_arr]
vecs = vecs[:, ind_arr]

vecs[:, 0] /= sum(vecs[:, 0])
Peq = vecs[:, 0]
```

SciPy does not guarantee any order when returning the eigenvalues, we sort them using `np.argsort`, which returns the indices of the sorted array of eigenvalues. The first column now corresponds to the correct eigenvector, which can be normalized to give $\mathbf{P}^{\text{eq}}$. The vectors are then orthonormalized with respect to the inner product (5) by the Gram-Schmidt process. Since array operations are by default elementwise in NumPy, the inner product can be implemented as `sum(v*w/Peq)`.

To calculate the coefficients, we leverage the array multiplication native to NumPy: when multiplying two arrays of different dimensionality, it keeps the dimensions of the larger one and multiplies the smaller one along the last n dimensions, where n is the difference in dimension between the two arrays. This can be used to efficiently compute otherwise expensive array operations. In the present case, we use this as:

$$\begin{pmatrix} (v_1)_1 & (v_2)_1 & \cdots \\ (v_1)_2 & (v_2)_2 & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix}^T * \begin{pmatrix} (\bar{\mathbf{W}}\mathbf{P}^{\text{eq}})_1 \\ (\bar{\mathbf{W}}\mathbf{P}^{\text{eq}})_2 \\ \vdots \end{pmatrix} = \begin{pmatrix} (v_1)_1 & (v_1)_2 & \cdots \\ (v_2)_1 & (v_2)_2 & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix} * \begin{pmatrix} (\bar{\mathbf{W}}\mathbf{P}^{\text{eq}})_1 \\ (\bar{\mathbf{W}}\mathbf{P}^{\text{eq}})_2 \\ \vdots \end{pmatrix} \quad (18)$$

$$= \begin{pmatrix} (v_1)_1(\bar{\mathbf{W}}\mathbf{P}^{\text{eq}})_1 & (v_1)_2(\bar{\mathbf{W}}\mathbf{P}^{\text{eq}})_2 & \cdots \\ (v_2)_1(\bar{\mathbf{W}}\mathbf{P}^{\text{eq}})_1 & (v_2)_2(\bar{\mathbf{W}}\mathbf{P}^{\text{eq}})_2 & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix}, \quad (19)$$

where $*$ denotes the NumPy multiplication. If we now divide the rows elementwise by $\mathbf{P}^{\text{eq}}$ and sum along them, we get the array

$$\begin{pmatrix} \langle \mathbf{v}_1 | \bar{\mathbf{W}}\mathbf{P}^{\text{eq}} \rangle \\ \langle \mathbf{v}_2 | \bar{\mathbf{W}}\mathbf{P}^{\text{eq}} \rangle \\ \vdots \end{pmatrix}. \quad (20)$$

Dividing this by the array of eigenvalues, we get an array containing the expansion coefficients of (13), with the notion that the first element should be omitted, since $\lambda_1 = 0$ and so the division is undefined. This can be avoided using slicing, and implemented as

```
p1coeffs = np.sum(vecs[:, 1:].T * np.matmul(w1, Peq)/Peq, axis=1) / vals[1:].
```

Next, we calculate the coefficients $A_{mn}^{(k)}$ and store them in a 3 dimensional array with indices `[n, m, :]`, such that the row `[n, m, :]` corresponds to the coefficients for the transition of

state n to state m . For this, we rewrite

$$A_{mn}^{(k)} = \begin{cases} \bar{W}_{mn}P_n^{\text{eq}} - \bar{W}_{nm}P_m^{\text{eq}} & k = 1, \\ -\frac{\langle \mathbf{v}_k | \bar{\mathbf{W}} \mathbf{P}^{\text{eq}} \rangle}{\lambda_k} (W_{mn}^{\text{eq}} v_{k,n} - W_{nm}^{\text{eq}} v_{k,m}) & k \geq 2, \end{cases} \quad (21)$$

and calculate the coefficients for each value of k .

Finally, (17) is calculated in a different file by a list comprehension:

```
cond = sum(
[coeffs[i, j, k] * (1 if k == 0 else (vals[k] / (1j * om - vals[k]))))
  for k in range(len(coeffs[0, 0, :]))]
).
```

3 Full code

```
import os
import numpy as np
import scipy.linalg as lin

def gram_schmidt(v_arr):
    """
    Orthogonalise a set of vectors given as the columns of an array using
    Gram-Schmidt procedure.

    :param v_arr: array of vectors to orthogonalise
    :return: array of orthogonalised vectors
    """
    # Orthogonalized, To Be Returned
    orthogonal = np.zeros(v_arr.shape)

    # At each step, take vector
    for i in range(len(v_arr)):
        v = v_arr[:, i]

        # Subtract off the "components" from current orthogonal set.
        for j in range(i):
            v -= sum(orthogonal[:, j]*v/Peq) * orthogonal[:, j]
        # Normalization
        v /= np.sqrt(sum(v*v/Peq))
        orthogonal[:, i] = v
    return orthogonal

def calculate_coeffs(weq, w1, pathname=None):
    global Peq
    n = len(weq[0, :]) # Infer matrix size from Weq

    vals, vecs = lin.eig(weq) # Get eigenvalues and
    eigenvecs, ensure real
    vals = np.real(vals)
    vecs = np.real(vecs)

    ind_arr = np.flip(np.argsort(vals)) # Sort eigenvalues in
    descending order and rearrange eigenvecs according

    vals = vals[ind_arr]
```

```

vecs = vecs[:, ind_arr]

vecs[:, 0] /= sum(vecs[:, 0])          # first eigenvector is Peq,
    normalize and rename
Peq = vecs[:, 0]

vecs = gram_schmidt(vecs)              # Orthonormalize
    eigenvectors

# Check validity of eigenvectors
for i in range(len(vecs)):
    if lin.norm(np.matmul(weq, vecs[:, i]) - vals[i] * vecs[:, i]) >= 10
        ** -14:
        print(lin.norm(np.matmul(weq, vecs[:, i]) - vals[i] * vecs[:, i]
            ))
        raise ValueError("Incorrect computation of eigenvectors")

# Calculate Coeffs of Pad expanded in eigenvecs
p1coeffs = np.sum(vecs[:, 1:].T * np.matmul(w1, Peq)/Peq, axis=1) / vals
    [1:]

# Calculate coefficients and store in array
coeffs = np.zeros([n, n, n])
for k in range(n):
    if k == 0:
        coeffs[:, :, k] = (w1 * Peq).T - w1 * Peq
    else:
        coeffs[:, :, k] = p1coeffs[k-1] * ((weq * vecs[:, k]).T - weq *
            vecs[:, k])

if pathname is not None:
    path = pathname.split("/")
    dir_path = "/" .join(path[:-1])

    os.makedirs(dir_path, exist_ok=True)

    np.savez(f"{pathname}.npz", coeffs=coeffs, vals=vals, vecs=vecs, weq
        =weq, w1=w1)

return coeffs, vals, vecs

```

References

- [1] N. G. van Kampen. *Stochastic processes in physics and chemistry*. 3rd ed. North-Holland personal library. OCLC: ocm81453662. Amsterdam ; Boston: Elsevier, 2007. ISBN: 978-0-444-52965-7.
- [2] Karel Proesmans, Bart Cleuren, and Christian Van Den Broeck. “Linear stochastic thermodynamics for periodically driven systems”. en. In: *Journal of Statistical Mechanics: Theory and Experiment* 2016.2 (Feb. 2016), p. 023202. ISSN: 1742-5468. DOI: 10.1088/1742-5468/2016/02/023202. URL: <https://iopscience.iop.org/article/10.1088/1742-5468/2016/02/023202> (visited on 02/13/2025).
- [3] Karel Proesmans and Carlos E. Fiore. “General linear thermodynamics for periodically driven systems with multiple reservoirs”. en. In: *Physical Review E* 100.2 (Aug. 2019),

p. 022141. ISSN: 2470-0045, 2470-0053. DOI: 10.1103/PhysRevE.100.022141. URL: <https://link.aps.org/doi/10.1103/PhysRevE.100.022141> (visited on 02/13/2025).

- [4] Bart Wijns and Bart Cleuren. “Impedance in Periodically driven Non-equilibrium Systems”. en. In: ().